CMINUS 编译器词法与语法分析实验报告

一、程序功能概述

1.1 词法分析功能

本程序实现了对 CMINUS 语言的完整词法分析，主要功能包括：

- 关键字识别：支持 int、float、if、else、while、return 六个关键字
- 常量处理：
  - 整数：支持十进制（123）、八进制（0123）、十六进制（0x1A）三种格式
  - 浮点数：支持多种格式（1.5、.5、1e10、1.5e-5 等）
- 标识符识别：字母开头，包含字母和数字的标识符
- 运算符与分隔符：支持算术运算、比较运算、逻辑运算等 20 余种符号
- 注释处理：支持 C 风格的 /* */ 多行注释和 C++ 风格的 // 单行注释
- 错误检测：能够识别无效的八进制、十六进制数及非法字符

1.2 语法分析功能

程序实现了基于 LL(1) 的语法分析器，支持以下语法结构：

- 程序结构：外部定义列表（变量声明 + 函数定义）
- 函数定义：包含函数头（标识符 + 参数列表）和函数体（复合语句）
- 变量声明：支持类型说明符（int/float）+ 变量列表，允许数组声明和初始化赋值
- 语句结构：表达式语句、复合语句、if-else 分支、while 循环、return 语句
- 表达式：支持赋值、算术运算、逻辑运算、关系运算、函数调用、数组访问等

1.3 抽象语法树（AST）生成

程序在语法分析过程中自动构建 AST，并以缩进形式打印输出。AST 节点包含节点类型、值和行号信息，便于后续语义分析阶段使用。

1.4 错误处理机制

- 词法错误（类型 A）：非法字符、无效常量格式、未闭合的注释
- 语法错误（类型 B）：缺失分号、括号不匹配、无效表达式等
- 错误信息包含错误类型、行号和具体描述，便于定位问题

二、编译与运行说明

2.1 编译环境要求

- 操作系统：Linux 或 macOS（需安装 GNU Flex 和 Bison）
- 编译器：GCC（支持 C99 标准）
- 工具依赖：
  - GNU Flex 2.6+
  - GNU Bison 3.0+

2.2 编译步骤

项目已提供完整的 Makefile，编译步骤如下：

```
# 进入项目目录
cd CompilerProject

# 使用 make 命令编译（推荐）
make

# 或手动执行编译命令
bison -d cminus.y
flex cminus.l
```

gcc -Wall -o cc cminus.tab.c lex.yy.c -lfl

2.3 清理命令

make clean

2.4 运行方式

./cc <输入文件>

2.5 测试用例

项目包含三个测试文件：

- test1.c：基本变量声明和函数定义
- test2.c：包含控制流语句的完整程序
- test3.c：数组和表达式测试

三、程序亮点与创新

3.1 多进制整数支持

词法分析器创新性地支持三种进制的整数表示：

- 十进制：常规数字序列（{DIGIT}+）
- 八进制：以 0 开头，仅包含 0-7（0[0-7]+）
- 十六进制：以 0x 或 0X 开头（0[xX][0-9a-fA-F]+）

关键设计：使用 strtol() 函数自动处理进制转换，代码简洁高效。

3.2 浮点数格式全面支持

格式	示例	正则表达式
标准小数	1.5	{DIGIT}+\.{DIGIT}*
省略整数部分	.5	\.{DIGIT}+
科学计数法（完整）	1.5e10	{DIGIT}+\.{DIGIT}*[eE][+-]?{DIGIT}+
科学计数法（省略整数）	.5e-5	\.{DIGIT}+[eE][+-]?{DIGIT}+
科学计数法（省略小数）	1e10	{DIGIT}+[eE][+-]?{DIGIT}+

3.3 完善的错误检测机制

词法错误检测：

- 八进制数中出现 8 或 9
- 十六进制数中出现非法字符（g-z）
- 科学计数法中指数部分缺失

语法错误检测：

- 使用 Bison 的错误恢复机制（error 符号）
- if-else 悬挂问题处理（通过 %nonassoc 指定优先级）
- 括号不匹配检测

3.4 AST 节点管理

设计了简洁的 AST 节点结构：

```
typedef struct Node {
    char name[50];          // 节点类型
    char value[100];        // 节点值
    int line;               // 行号
    struct Node *children[10]; // 子节点（最多 10 个）
    int child_count;        // 子节点数量
} Node;
```

提供了完整的节点操作函数：create_node()、add_child()、print_tree()、free_tree()。

3.5 代码结构优化

- 模块化设计：词法分析与语法分析分离，便于维护和扩展
- 内存管理：AST 构建完成后自动释放内存，避免内存泄漏
- 错误计数：使用全局变量追踪错误数量，便于判断分析结果

四、测试结果示例

4.1（test1.c）

```
lyx20240704@lyx20240704-VMware-Virtual-Platform:~/CompilerProject$ ./cc test1.c
Error type A at Line 4: Mysterious character "~"
```

4.2（test2.c）

```
lyx20240704@lyx20240704-VMware-Virtual-Platform:~/CompilerProject$ ./cc test2.c
Error type B at Line 5: Missing "]"
Error type B at Line 6: Missing ';'
```

4.3（test3.c）

```
lyx20240704@lyx20240704-VMware-Virtual-Platform:~/CompilerProject$ ./cc test3.c
Program (1)
  ExtDefList (1)
    ExtDef (1)
      Specifier (1)
        TYPE: int
      FunDec (1)
        ID: inc
        LP (1)
        RP (1)
      CompSt (5)
        LC (5)
        DefList (3)
          Def (3)
            Specifier (3)
              TYPE: int
            VarDec (3)
              ID: i
            SEMI (3)
        StmtList (4)
          Stmt (4)
            Exp (4)
              Exp (4)
                ID: i
              ASSIGNOP (4)
              Exp (4)
                Exp (4)
                  ID: i
```

```
              PLUS (4)
              Exp (4)
                INT: 1
          SEMI (4)
      RC (5)
```